

인공지능 데이터 전처리 결과서

1. 문서 개요

본 결과서는 League of Legends 사용자 이탈 예측 프로젝트에서 모델 학습에 사용되는 데이터가 어떤 기준으로 수집·정리·선별·가공되었는지 기록한 산출물이다. 특히 기준일 이후 정보를 입력 피처에서 배제하여 시간 누수를 방지하고, 저활동·신규 유입 사용자를 분리하여 이탈 예측이 가능한 진성 활동 고객 코호트를 구성하는 과정에 초점을 둔다.

분석 기준일	2026-08-01
전체 수집 사용자	9,879 명
최종 진성 활동 고객	2,381 명
30 일 무경기 라벨	363 명 (15.25%)
입력 관찰 범위	최대 기준일 이전 90 일 + 최근 20 경기 요약
라벨 관찰 범위	기준일 이후 30 일

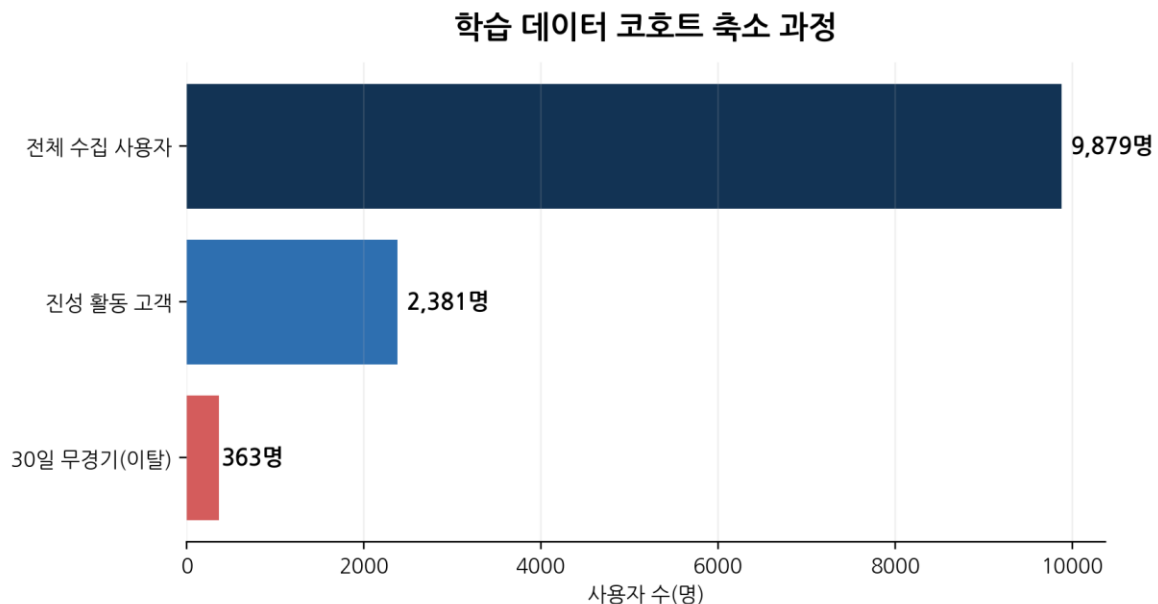


그림 1. 전체 수집 사용자에서 학습 코호트까지의 축소 과정

2. 원천 데이터 및 데이터 파이프라인

1. 여러 수집 배치에서 사용자와 경기 이력을 통합한다.
2. team_id + player_id 조합을 내부 조회 키로 사용하여 사용자 중복과 팀 간 충돌을 확인한다.
3. 경기 목록, 경기 상세, 게임 모드 및 경기 시점을 정리한다.
4. 기준일 이전 활동 이력을 이용해 사용자 단위 집계 피처를 생성한다.
5. 기준일 이후 30 일의 추가 경기 유무를 확인해 이탈 라벨을 생성한다.

핵심 원천	Riot Games API 기반 사용자/경기 이력
분석 단위	사용자 단위
시퀀스 단위	경기 이벤트 단위
조회 식별자	team_id:player_id
식별자 학습 사용	사용자 ID 와 team_id 는 학습 피처에서 제외

해석 주의 현재 데이터 화면의 player_id 가 Riot PUUID 임을 확인할 수 있는 메타정보가 없으므로 문서에서는 “분석용 사용자 ID”로 표기한다.

3. 코호트 선정 및 라벨 정의

3.1 진성 활동 고객 선정 규칙

구분	조건	의도
포함	games_prev30d $\geq$ 4	직전 기간에도 충분한 활동이 있었던 사용자
포함	games_30d > 0	기준일 직전 30 일 내 실제 플레이가 있는 사용자
제외	둘 중 하나라도 미충족	신규·저활동·우연 유입 사용자 분리

3.2 이탈 라벨 정의

본 프로젝트에서 이탈(churn=1)은 "기준일 이후 30 일 동안 대상 게임 모드의 추가 경기 기록이 없는 상태"로 정의한다. 이는 Riot Games 의 공식 탈퇴·휴면 기준이 아니며, 로그인 여부나 계정 해지를 의미하지 않는다.

시간 누수 방지를 위한 관찰 구간 분리

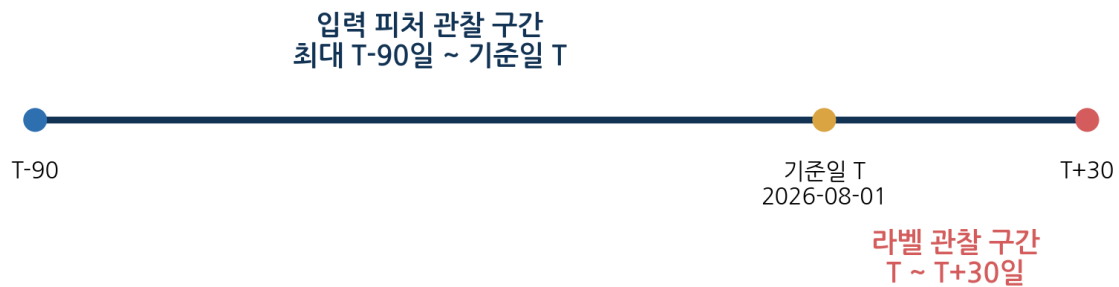


그림 2. 입력 관찰 구간과 이탈 라벨 구간의 시간 분리

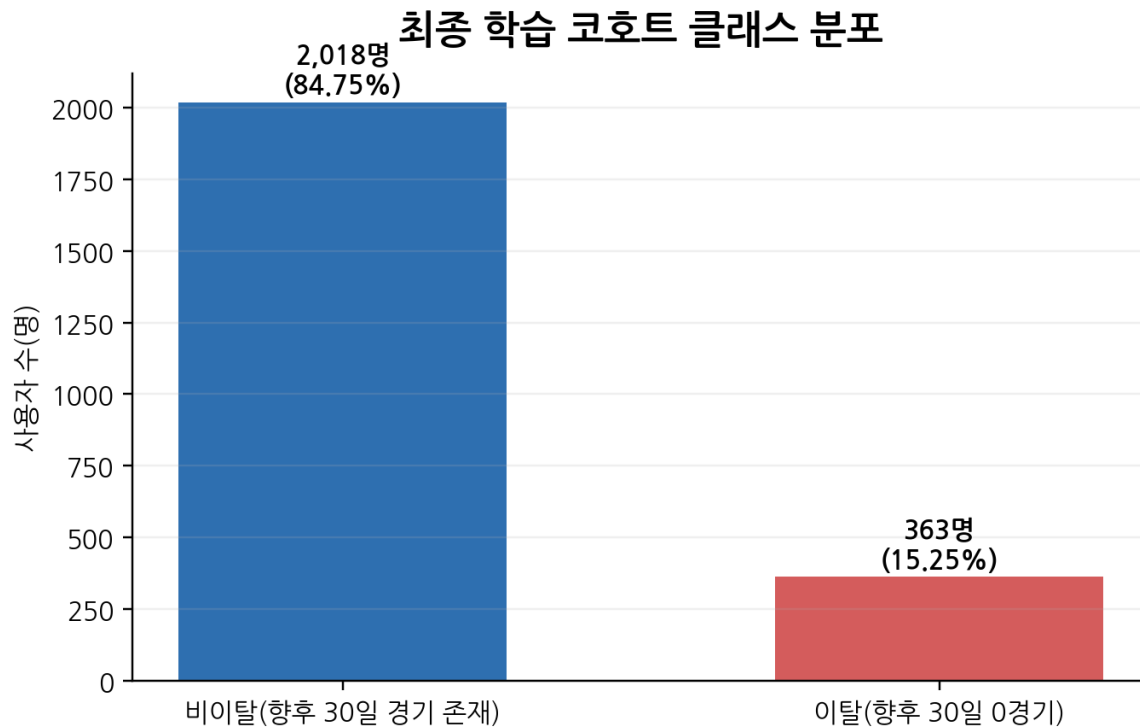


그림 3. 최종 학습 코호트의 클래스 불균형

4. 전처리 및 품질 관리 결과

점검 항목	처리 내용	결과
중복 관리	team_id + player_id, matchId 기준으로 동일 사용자·경기 중복을 확인	완료
수집 상태 분리	오류·빈 응답·실제 무경기 상태를 구분	완료
결측 처리	모델 비교 시 각 학습 Fold 에서 중앙값을 적합하여 결측을 보완	완료
시간 누수 방지	기준일 이후 정보는 피처 생성에서 제외	완료
스키마 검증	컬럼 타입·범위·결측 처리 기준을 점검	완료
식별자 제외	player_id, team_id 를 모델 입력에서 제외	완료
티어 제외	2,381 명 중 일부(792 명)만 매칭되고 수집 시점 불명확하여 입력에서 제외	완료

4.1 전처리 상세 수행 절차

표 형태의 점검 결과만으로는 실제 데이터가 어떻게 학습용 행으로 변환되는지 파악하기 어렵기 때문에, 본 절에서는 전처리 순서를 데이터 흐름 기준으로 상세히 기술한다. 핵심 원칙은 (1) 동일 사용자·경기 중복 제거, (2) 기준일 이전/이후 시간 분리, (3) 진성 활동 고객 코호트 선정, (4) 사용자 단위 33 개 집계 피처 생성, (5) Fold 내부 결측치·불균형 처리이다.

전처리 상세 처리 흐름



그림 4. 원천 경기 데이터에서 최종 학습 행까지의 전처리 흐름

코드 표기 기준 아래 코드는 본 결과서에 기록된 전처리 규칙을 이해하기 쉽게 축약한 재현 예시이다. 실제 저장소의 함수명·원천 컬럼명과 1:1로 동일하다고 가정하지 않으며, 코호트 조건·시간 구간·피처 구성·Fold 처리 원칙은 현재 결과서 기준을 따른다.

4.2 중복 제거와 시점 정규화

먼저 사용자 단위 중복과 경기 단위 중복을 제거한다. 사용자 식별은 team_id 와 player_id 의 조합을 사용하고, 경기 이력은 matchId 를 기준으로 동일 경기가 중복 적재되지 않도록 정리한다. 이후 경기 시각을 datetime 형식으로 변환하여 기준일과의 차이를 계산할 수 있는 상태로 만든다. 날짜 파싱에 실패한 행은 시간 기반 피처에 사용할 수 없으므로 별도 점검 대상이 된다.

```
users = users.drop_duplicates(["team_id", "player_id"])
matches = matches.drop_duplicates(["matchId"])

matches["game_datetime"] = pd.to_datetime(
    matches["game_datetime"], errors="coerce", utc=True
)
matches = matches.dropna(subset=["game_datetime"])
```

- 사용자 키와 경기 키를 분리하여 관리함으로써 동일 사용자가 여러 경기 행을 갖는 정상적인 1:N 구조와 진짜 중복 행을 구분한다.
- 문자열 날짜를 그대로 비교하지 않고 datetime 으로 변환하여 최근 7 일·30 일·90 일 및 향후 30 일 구간을 일관되게 계산한다.

4.3 기준일 분리, 코호트 선정 및 이탈 라벨 생성

기준일 T=2026-08-01 을 경계로 입력 피처 생성에 사용할 과거 데이터와 정답 라벨 생성에 사용할 미래 데이터를 물리적으로 분리한다. 피처 계산에는 T 이전 정보만 사용하고, T 이후 30 일 데이터는 오직 churn 정답을 만들기 위해서만 사용한다. 이 구조가 시간 누수를 막는 핵심이다.

```
cutoff = pd.Timestamp("2026-08-01", tz="UTC")

past = matches[matches["game_datetime"] < cutoff].copy()
future30 = matches[
    (matches["game_datetime"] >= cutoff)
    & (matches["game_datetime"] < cutoff + pd.Timedelta(days=30))
].copy()
```

전체 수집 사용자 9,879 명을 그대로 학습 대상으로 사용하지 않고, 기준일 이전에도 일정 수준의 활동이 확인된 사용자만 진성 활동 고객으로 남긴다. 결과서에 기록된 규칙은 games_prev30d $\geq$ 4 이면서 games_30d > 0 인 사용자이며, 이 조건을 통과한 최종 학습 코호트는 2,381 명이다.

```

core = user_features[
    (user_features["games_prev30d"] >= 4)
    & (user_features["games_30d"] > 0)
].copy()

future_cnt = (
    future30.groupby(["team_id", "player_id"])
    .size().rename("future_games_30d").reset_index()
)

core = core.merge(future_cnt, on=["team_id", "player_id"], how="left")
core["future_games_30d"] = core["future_games_30d"].fillna(0)
core["churn"] = (core["future_games_30d"] == 0).astype("int8")

```

- $\text{games_prev30d} \geq 4$ 조건은 직전 비교 구간에도 일정 수준 이상 플레이가 있었는지 확인하여 신규·일회성 유입 사용자를 줄이기 위한 장치다.
- $\text{games_30d} > 0$ 조건은 기준일 직전 30 일에 실제 플레이가 있었던 사용자만 남겨, 이미 오래 활동하지 않은 계정을 예측 대상으로 섞는 문제를 줄인다.
- 향후 30 일 경기 수가 0 이면 $\text{churn}=1$, 1 회 이상이면 $\text{churn}=0$ 으로 정의한다. 최종 분포는 이탈 363 명(15.25%), 비이탈 2,018 명(84.75%)이다.

4.4 사용자 단위 피처 집계

경기 원천 데이터는 한 사용자가 여러 행을 갖는 이벤트 데이터이므로 그대로 분류 모델에 넣지 않고, 기준일 이전 경기들을 사용자별로 집계해 1 사용자=1 행 형태로 변환한다. 최종 aggregate33 은 활동 빈도·공백, 최근 경기 성과, 분당 효율, 챔피언 다양성, 게임 모드 행동을 함께 표현한다.

피처 범주	개수	주요 내용
활동량·공백	10 개	최근 7/30/90 일 경기 수, 직전 30 일, 활동일 수, 마지막 경기 경과일, 평균·최대 경기 간격 등
최근 경기 성과	8 개	최근 20 경기 승률·KDA·킬·데스·어시스트, 연패, 최근 승률 변화, 평균 게임 시간
분당 효율	4 개	분당 골드·CS·피해량·시아 점수
챔피언 다양성	2 개	최근 20 경기 고유 챔피언 수, 최다 챔피언 사용 비율
게임 모드 행동	9 개	모드 종류·전환 빈도와 솔랭/자랭/일반/칼바람/아레나/기타 모드 비율

예를 들어 최근 20 경기 통계는 기준일 이전 경기만 시간순으로 정렬한 뒤 사용자별 마지막 20 개 행을 선택하고 평균·비율을 계산한다. 최근 30 일 모드 비율은 해당 사용자의 30 일 경기 수를 분모로 두어 각 모드의 플레이 비중을 만든다.

```

keys = ["team_id", "player_id"]
last20 = (
    past.sort_values("game_datetime")
        .groupby(keys, group_keys=False)
        .tail(20)
)

agg20 = last20.groupby(keys).agg(
    winrate_20=("win", "mean"),
    avg_kills_20=("kills", "mean"),
    avg_deaths_20=("deaths", "mean"),
    avg_assists_20=("assists", "mean"),
    unique_champions_20=("champion", "nunique"),
)

```

```

past30 = past[
    past["game_datetime"] >= cutoff - pd.Timedelta(days=30)
].copy()

mode_cnt = past30.groupby(keys + ["game_mode"]).size()
mode_ratio = mode_cnt / mode_cnt.groupby(level=[0, 1]).transform("sum")

```

중요 피처 계산 기준 시점은 항상 기준일 이전이다. 예를 들어 `days_since_last_game` 역시 기준일 T와 마지막 과거 경기 시각의 차이로 계산해야 하며, T 이후 경기 시각을 참조하면 라벨 정보가 입력에 섞이는 누수가 발생한다.

4.5 모델 입력 정리: 식별자 제외와 결측치 처리

집계가 끝난 뒤 `player_id`와 `team_id`는 조회·화면 연결용 키로만 보존하고 모델의 설명변수 X에서는 제외한다. 티어 정보도 일부 사용자만 매칭되고 수집 시점이 명확하지 않아 최종 입력에서 제외한다. 모델 입력은 선택된 33개 수치형 피처만 사용한다.

```

FEATURES = aggregate33_columns
X = core[FEATURES].copy()
y = core["churn"].astype(int)

# identifiers remain outside model input
assert "player_id" not in X.columns
assert "team_id" not in X.columns

```

결측치는 전체 데이터를 한 번에 채우지 않는다. 검증 Fold의 분포가 학습 Fold로 새어 들어가지 않도록 매 Fold마다 train 부분에서만 중앙값을 계산하고, 그 중앙값으로 train과 validation을 각각 보완한다. 이는 전처리 단계에서도 검증 데이터 독립성을 유지하기 위한 처리다.

```

skf = StratifiedKFold(
    n_splits=5, shuffle=True, random_state=42
)

for tr_idx, va_idx in skf.split(X, y):
    X_tr, X_va = X.iloc[tr_idx].copy(), X.iloc[va_idx].copy()
    y_tr, y_va = y.iloc[tr_idx], y.iloc[va_idx]

    med = X_tr.median(numeric_only=True)
    X_tr = X_tr.fillna(med)
    X_va = X_va.fillna(med)

    pos_weight = (y_tr == 0).sum() / max((y_tr == 1).sum(), 1)

```

- Fold 별 중앙값: validation 데이터의 통계량을 전처리 기준 산정에 사용하지 않는다.
- Stratified 5-Fold: 이탈 비율 15.25%의 클래스 불균형을 각 Fold 에서 유사하게 유지한다.
- class weight: 양성(이탈) 표본이 적기 때문에 학습 Fold 별 양성/음성 빈도를 반영해 이탈 클래스의 학습 영향력을 보완한다.

최종 확인 항목 전처리 완료 데이터는 사용자당 1 행, 33 개 입력 피쳐, churn 1 개 타깃으로 구성한다. 모든 피쳐는 T 이전 데이터로만 생성하고, T 이후 30 일 데이터는 churn 라벨 생성 외에는 사용하지 않는다.

5. 피쳐 엔지니어링 결과

최종 선택 피쳐셋은 aggregate33 이며 총 33 개 집계 피쳐를 사용한다. 별도로 7 개의 추가

파생변수(recent_activity_ratio 등)를 실험한 engineered40 후보와 SHAP 상위 15 개만 사용하는 shap15 후보도 비교했으나 최종 구성은 aggregate33 + class_weight_balanced 로 선정되었다.

No.	피쳐명	설명
1	games_7d	최근 7 일 경기 수
2	games_30d	최근 30 일 경기 수
3	games_90d	최근 90 일 경기 수
4	games_prev30d	직전 비교 30 일 경기 수
5	activity_change_30d	최근 30 일 활동량 변화
6	active_days_30d	최근 30 일 실제 활동일 수
7	days_since_last_game	마지막 경기 이후 경과일

8	avg_gap_30d	최근 30 일 평균 경기 간격
9	max_gap_30d	최근 30 일 최대 경기 간격
10	last_gap	가장 최근 경기 간격
11	winrate_20	최근 20 경기 승률
12	avg_kda_20	최근 20 경기 평균 KDA
13	avg_kills_20	최근 20 경기 평균 Kill
14	avg_deaths_20	최근 20 경기 평균 Death
15	avg_assists_20	최근 20 경기 평균 Assist
16	losing_streak	최근 연속 패배 길이
17	winrate_change_10	최근 10 경기 기준 승률 변화
18	avg_game_duration_20	최근 20 경기 평균 게임 시간

표 1. 최종 입력 피쳐 1~18

No.	피쳐명	설명
19	avg_gold_per_min_20	최근 20 경기 분당 골드
20	avg_cs_per_min_20	최근 20 경기 분당 CS
21	avg_damage_per_min_20	최근 20 경기 분당 피해량
22	avg_vision_per_min_20	최근 20 경기 분당 시야 점수
23	unique_champions_20	최근 20 경기 사용 챔피언 수
24	top_champion_ratio_20	최근 20 경기 최다 챔피언 사용 비율
25	unique_modes_30d	최근 30 일 게임 모드 종류 수
26	mode_switch_rate_20	최근 20 경기 모드 전환 빈도
27	ranked_solo_ratio_30d	최근 30 일 솔로랭크 비율

28	ranked_flex_ratio_30d	최근 30 일 자유랭크 비율
29	normal_ratio_30d	최근 30 일 일반게임 비율
30	aram_ratio_30d	최근 30 일 칼바람 비율
31	arena_ratio_30d	최근 30 일 아레나 비율
32	rotating_ratio_30d	최근 30 일 로테이션 모드 비율
33	other_ratio_30d	최근 30 일 기타 모드 비율

표 2. 최종 입력 피쳐 19~33

6. 최종 데이터셋 요약

학습 대상	2,381 명
양성 클래스(이탈)	363 명
음성 클래스(비이탈)	2,018 명
양성 비율	15.25%
최종 입력 피쳐 수	33 개
결측 처리	Fold 별 train 중앙값
불균형 대응	학습 Fold 별 양성/음성 역빈도 기반 class weight
누수 방지	모든 학습 피쳐는 기준일 이전 정보만 사용

코드 예시 1. 사용자·경기 중복 및 경기 시각 정규화

코드 예시 2. 기준일 이전 입력 구간과 이후 30 일 라벨 구간 분리

코드 예시 3. 진성 활동 고객 필터와 30 일 무경기 라벨 생성

코드 예시 4. 최근 20 경기 기반 사용자 집계 피쳐

코드 예시 5. 최근 30 일 게임 모드 비율 생성

코드 예시 6. 최종 모델 입력 X와 타깃 y 구성

코드 예시 7. Fold 내부 중앙값 보완 및 양성 클래스 가중치 계산